

Formation FPGA :

TP n°4 Partie 1 : Pilotage de LED et mémoire

Yassine Aouini

Objectifs du TP :

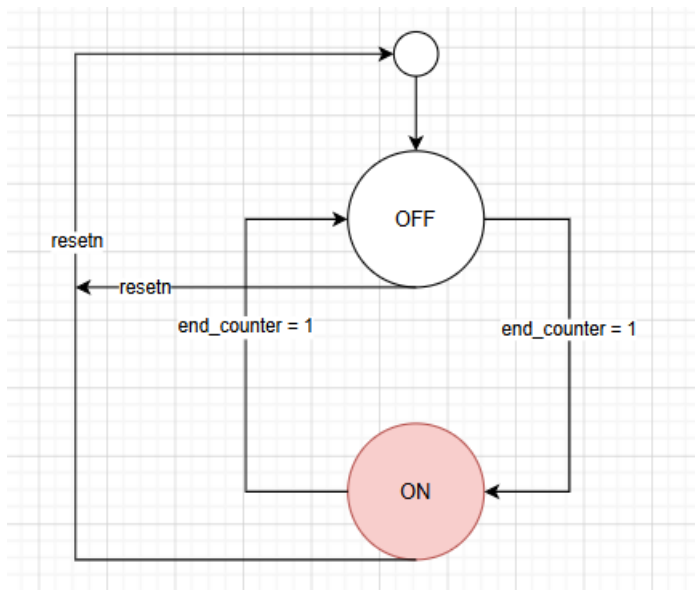
- Piloter une LED RGB avec des boutons
- Mettre en place une séquence de couleurs
- Manipuler des composants mémoire (FIFO)

Objectifs du TP :	1
Question 1 :	2
Question 2 :	3
Question 3 :	3
Question 4 :	3
Question 5 :	4
Question 6 :	4
Question 7 :	6
Question 8 :	7
Question 9 :	8
Question 10 :	8
Question 11 :	8
Question 12 :	9
Question 13 :	9
Question 14 :	11
Question 15 :	12

Question 1 :

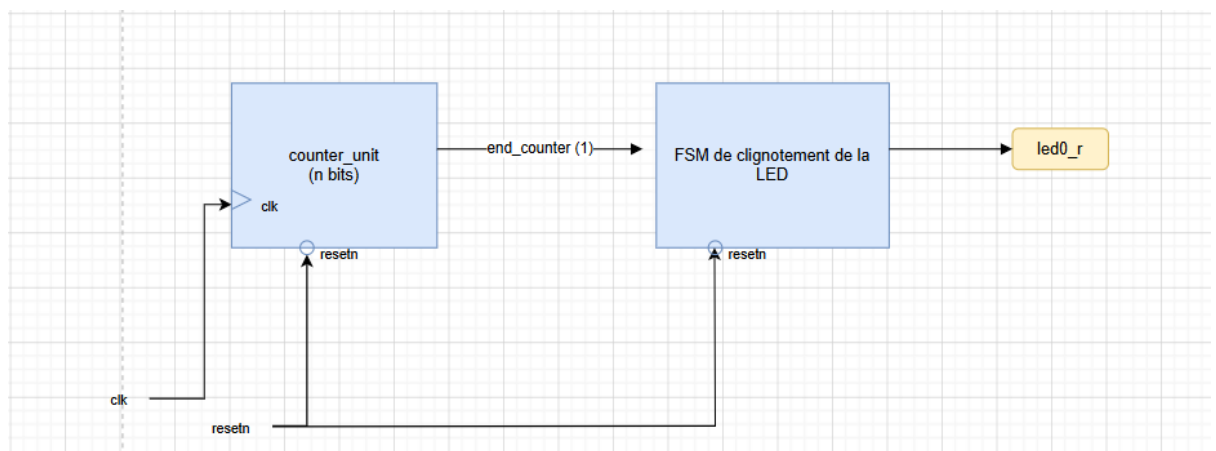
Construisons le schéma RTL permettant de faire clignoter une LED. Pour cela nous aurons besoin du module `counter_unit` précédemment utilisé au TP2 et d'une machine à état.

Le schéma de la machine à état est le suivant :



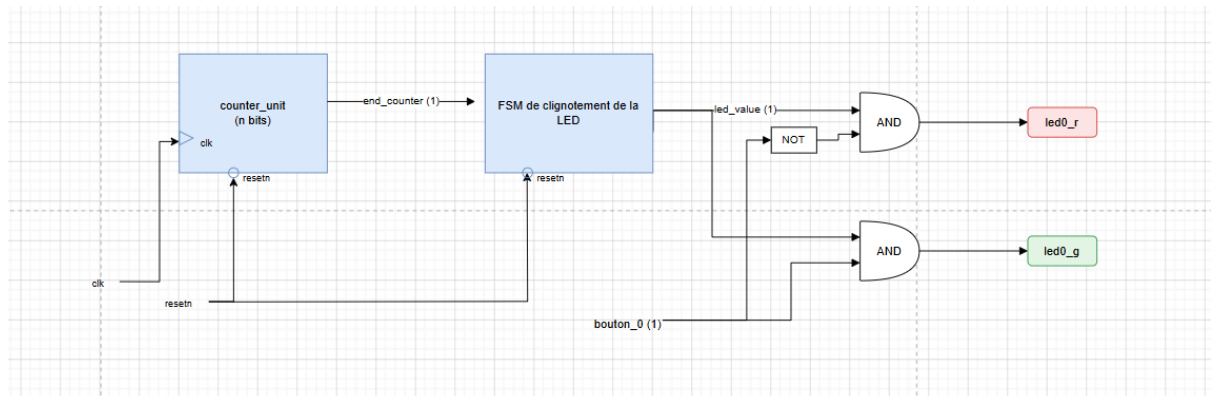
La LED clignote sur réception du signal `end_counter` et peut être remise à l'état initial en activant le `reset`.

Nous allons ajouter cette machine à état dans l'architecture RTL complète. Le choix de la couleur est arbitraire, ici ce sera le rouge.



Question 2 :

Nous allons modifier le schéma ci-dessus :



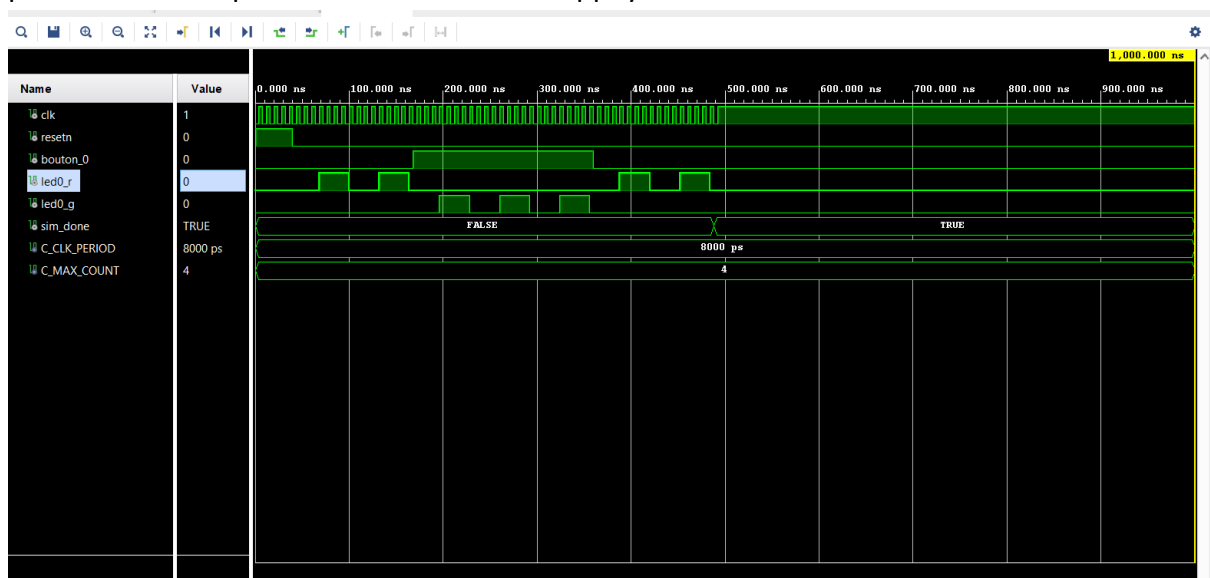
Si la LED est allumée et que le bouton_0 est appuyé, la couleur sera verte.

Si la LED est allumée mais que le bouton_0 est relâché, la couleur sera rouge.

Question 3 :

Question 4 :

Si le bouton est pressé pendant plus d'un cycle d'horloge alors la LED clignotera en vert pendant toute la période où le bouton est appuyé.



Question 5 :

On veut que la LED ne clignote qu'une fois même si le bouton reste maintenu. La condition pour réaliser cela doit être d'allumer la LED en vert en cas de détection de front montant de bouton_0.

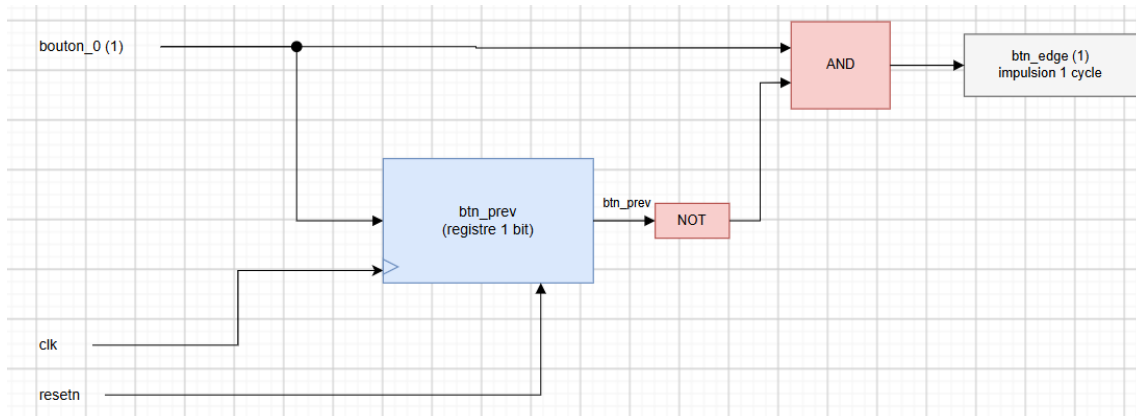
En effet, sur toute la période où le bouton reste appuyé, on ne détecte qu'une seule fois le front montant de bouton_0.

Question 6 :

Une manière de représenter le front montant d'un signal est la suivante :

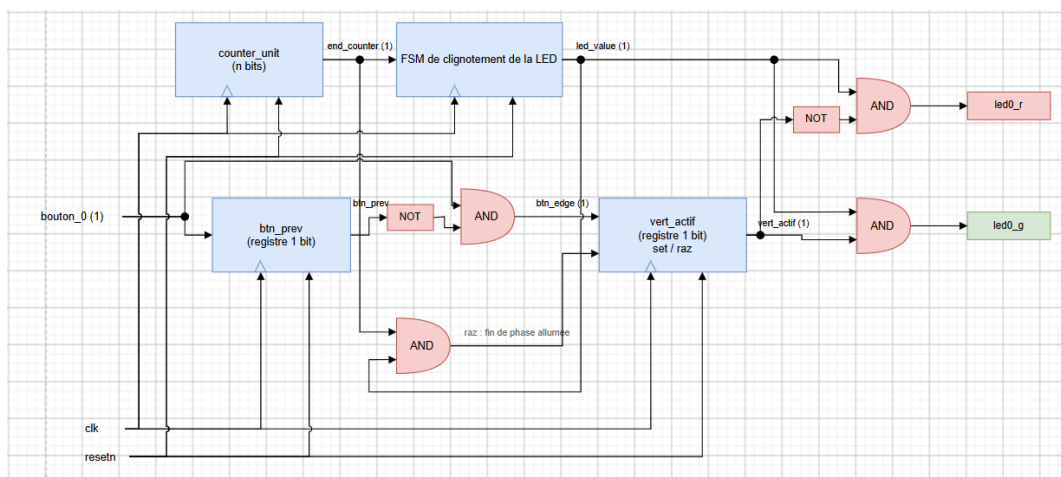
Le signal vaut 1 maintenant **ET** il valait 0 au coup d'horloge précédent.

Pour représenter ceci sur un schéma RTL, on va se servir d'un registre pour stocker la valeur précédente du signal bouton_0. Ce sera notre détecteur de front.



La détection de front n'est pas suffisante pour notre cas, car un front montant ne dure qu'un coup d'horloge. On modifie donc le schéma complet afin d'y ajouter un élément de stockage de l'information sur la couleur de la LED, le registre vert_actif.

Le schéma final :



L'association de counter_unit et de la FSM de clignotement de la LED permet d'avoir un clignotement totalement indépendant de l'appui sur le bouton.

Les registres btn_prev et vert_actif vont nous permettre d'établir le changement de la couleur de la LED du rouge vers le vert comme indiqué dans l'énoncé. Le détecteur de front nous assure de n'allumer la LED verte qu'une fois pour chaque appui sur le bouton.

Le registre vert_actif présente deux entrées : la sortie du détecteur de front pour le "set" et l'association de end_counter et de led_value pour le "raz". En clair, une fois la LED verte allumée, le registre se réinitialise à la fin du clignotement de la LED.

Question 7 :

On peut vérifier plusieurs points par simulation :

- 1) La cadence de clignotement de la LED reste inchangée. L'appui sur le bouton et le changement de couleur ne modifie pas la manière dont la LED clignote.
- 2) L'apparition d'un seul clignotement vert en cas d'appui sur le bouton. Pour cela, on va tester plusieurs cas d'usage afin de s'assurer de la fiabilité du programme.

Ainsi, on peut voir que notre système est passé d'un fonctionnement selon l'état du bouton à un fonctionnement selon l'évènement d'appui du bouton. C'est le résultat souhaité.



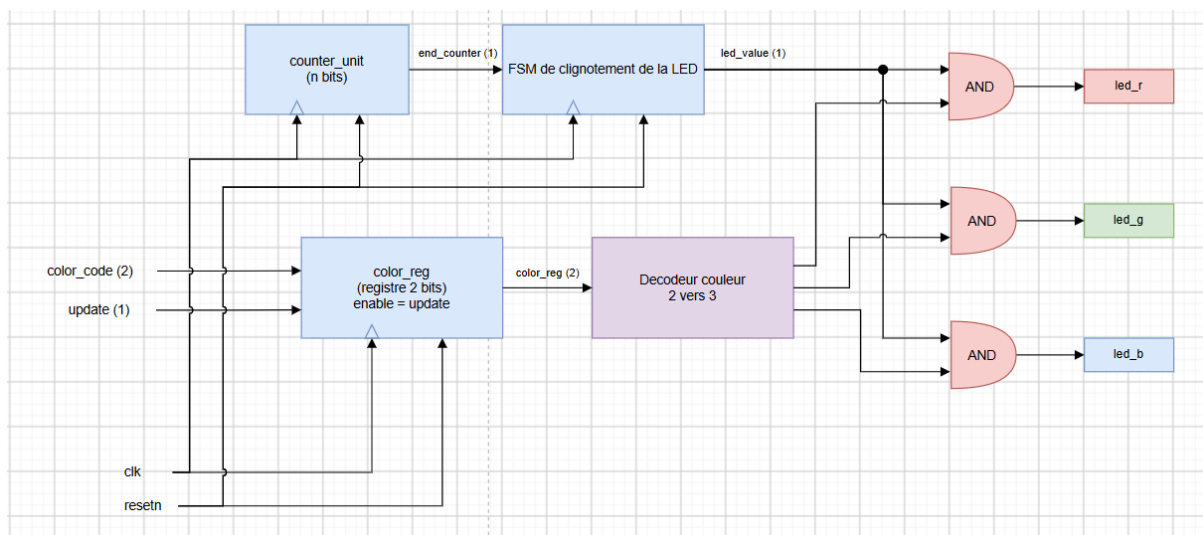
- Pour nous assurer du maintien de la cadence de la LED, on compare led_value à led_union, qui est l'union de la led rouge et de la led verte.
- u_prev et g_prev représentent led_union et led0_g déphasés d'un coup d'horloge, afin de pouvoir détecter les fronts montants respectifs.
- Check_on nous sert à calibrer le moniteur de cadence sur le premier clignotement plutôt que sur le début de la simulation, dans le but de détecter toute anomalie au niveau de la longueur des signaux.

Plusieurs scénaris sont mis en oeuvre pour ce test_bench.

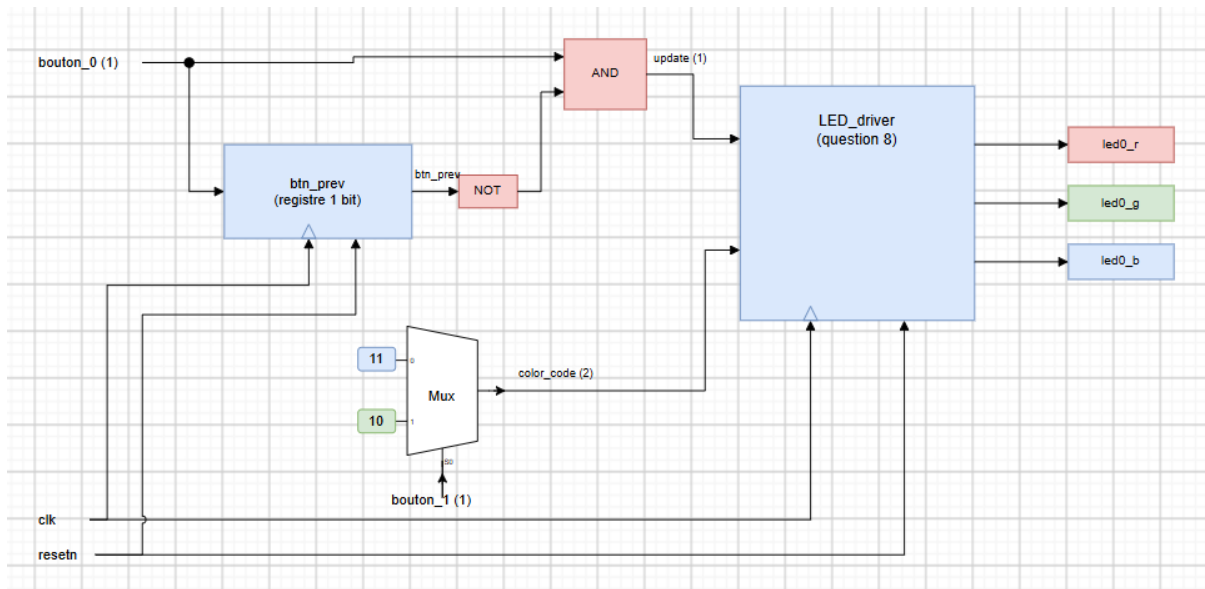
- Sans appui sur le bouton, la led rouge clignote normalement
- Avec un appui prolongé sur le bouton, un seul clignotement de la led verte est observé, comme souhaité.
- Avec un appui bref suite au relâchement du bouton, la led clignote en vert une fois de plus.
- On observe que la led verte s'allume même en cas d'appui lors de la phase d'allumage.
- Enfin, les appuis successifs suivant sont décalés chacun d'un coup d'horloge par rapport au précédent, afin de s'assurer du bon fonctionnement du système à tout instant.

Question 8 :

Voici le module permettant le pilotage de la LED RGB, le code couleur est respecté grâce aux portes AND en parallèle. Ce module sera plus tard appelé LED_Driver.



Question 9 :



Pour répondre aux exigences du système, nous allons réutiliser notre module détecteur de front afin de nous assurer que le maintien du bouton ne maintienne pas le signal update à 1.

Le traitement de la valeur de color_code va être pris en charge par un élément MUX.

Question 10 :

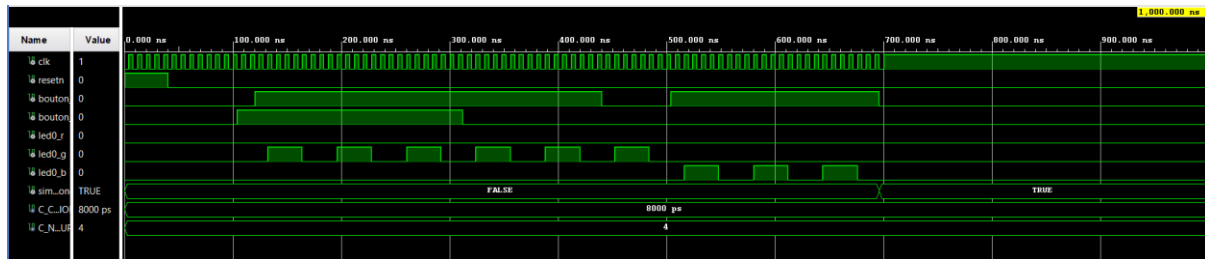
(Cf. Fichier vhd top_led_driver)

Question 11 :

Dans ce test bench, nous pouvons tester les exigences de la question 9.

- Le signal update doit recevoir 1 uniquement lorsque le bouton_0 vient d'être appuyé, maintenir le bouton enfoncé ne doit pas maintenir le signal update à 1
- Le signal color_code doit recevoir soit le code couleur « vert » si le bouton_1 est pressé, il doit recevoir le code couleur « bleu » sinon
- Les LED R, G et B sont connectées avec les couleurs respectives de la LED_0

Question 12 :



On observe sur le chronogramme les différents cas d'utilisation.

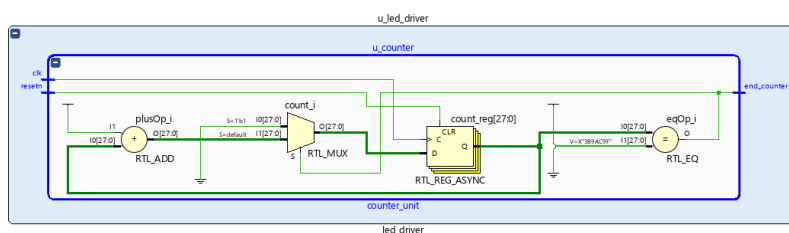
- À l'initialisation, aucune LED n'est allumée car le code couleur vaut 00.
- Tant que le bouton 0 n'est pas appuyé, le signal update n'est pas émis donc le changement de couleur n'est pas pris en compte bien que le bouton 1 soit appuyé.
- Une fois le bouton 0 appuyé, la LED s'allume en vert conformément au fonctionnement souhaité.
- Le bouton 1 est relâché et le bouton 0 est maintenu. Comme il n'y a pas eu de nouveau front montant du bouton 0, la LED continue de clignoter en vert.
- On relâche le bouton 0 puis on réappuie dessus, cette fois la LED clignote en bleu.

Question 13 :

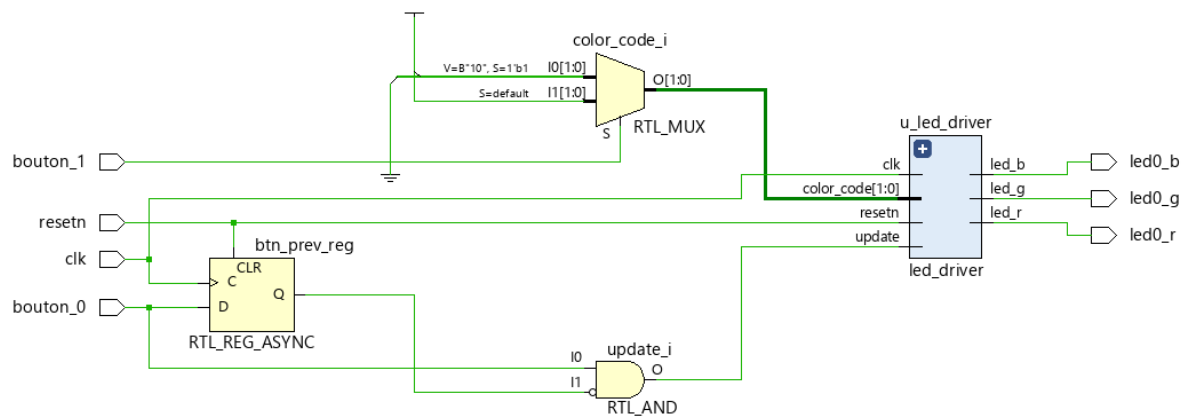
Site Type	Used	Fixed	Available	Util%
Slice LUTs*	39	0	17600	0.22
LUT as Logic	39	0	17600	0.22
LUT as Memory	0	0	6000	0.00
Slice Registers	32	0	35200	0.09
Register as Flip Flop	32	0	35200	0.09
Register as Latch	0	0	35200	0.00
F7 Muxes	0	0	8800	0.00
F8 Muxes	0	0	4400	0.00

En observant les ressources utilisées, on trouve un nombre de 32 bascules pour notre système.

Notre module counter_unit du TP2 demande 28 bascules pour son usage.

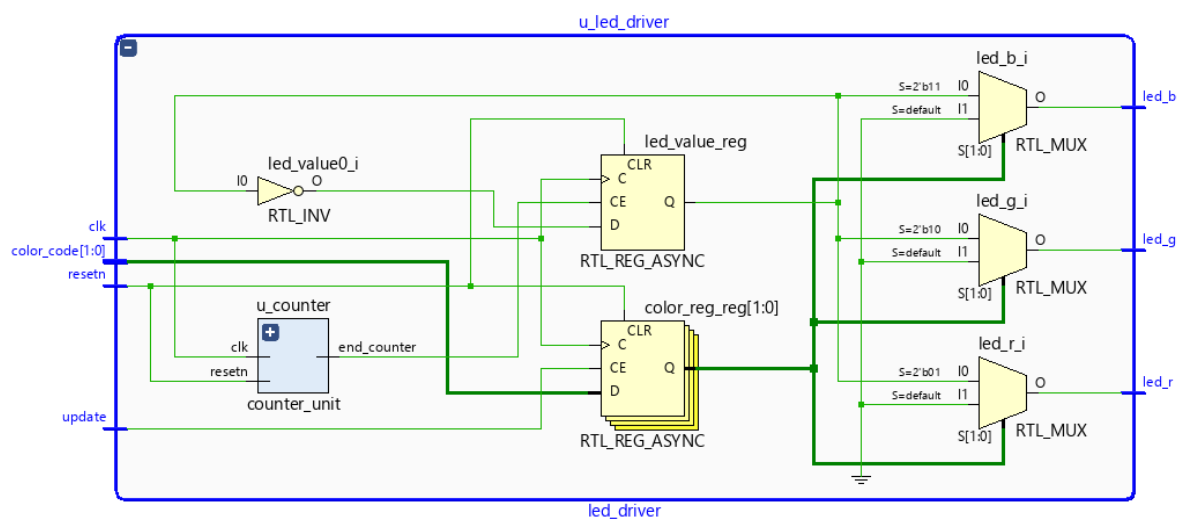


Une bascule est utilisée par le module de détection de front montant.



Une bascule est utilisée pour le stockage de la valeur led_value.

Enfin, deux bascules permettent le stockage du code couleur de la LED.



On obtient ainsi 32 bascules au total, ce qui est cohérent avec le rapport de synthèse.

Question 14 :

Setup		Hold		Pulse Width	
Worst Negative Slack (WNS):	3,152 ns	Worst Hold Slack (WHS):	0,263 ns	Worst Pulse Width Slack (WPWS):	3,500 ns
Total Negative Slack (TNS):	0,000 ns	Total Hold Slack (THS):	0,000 ns	Total Pulse Width Negative Slack (TPWS):	0,000 ns
Number of Failing Endpoints:	0	Number of Failing Endpoints:	0	Number of Failing Endpoints:	0
Total Number of Endpoints:	31	Total Number of Endpoints:	31	Total Number of Endpoints:	33

All user specified timing constraints are met.

Les valeurs de TNS et de THS étant nulles, aucune contrainte de timing n'est violée.

Les marges positives en setup comme en hold nous indiquent que :

- La marge de setup de 3,363 ns garantit que la logique combinatoire s'établit largement avant le front d'horloge suivant.
- La marge de hold de 0,294 ns garantit que les données ne changent pas trop tôt après le front.

La marge de setup correspond à une fréquence maximale de fonctionnement de :

$$F_{\max} = 1 / (8,000 - 3,152) = 1 / 4,848 \text{ ns} \approx \mathbf{206,3 \text{ MHz}}$$

Le design pourrait donc fonctionner bien au-delà de la fréquence de 125 MHz imposée par la carte.

Site Type	Used	Fixed	Available	Util%
Slice LUTs	39	0	17600	0.22
LUT as Logic	39	0	17600	0.22
LUT as Memory	0	0	6000	0.00
Slice Registers	32	0	35200	0.09
Register as Flip Flop	32	0	35200	0.09
Register as Latch	0	0	35200	0.00
F7 Muxes	0	0	8800	0.00
F8 Muxes	0	0	4400	0.00

Les décomptes de LUT et de bascules demeurent inchangés par rapport à la synthèse : l'optimisation d'implémentation n'a trouvé aucune simplification supplémentaire, le design ayant déjà atteint sa forme minimale.

```

Max Delay Paths
-----
Slack (MET) :          3.152ns  (required time - arrival time)
Source:          u_led_driver/u_counter/count_reg[6]/C
                  (rising edge-triggered cell FDCE clocked by sys_clk_pin  (rise@0.000ns fall@4.000ns period=8.000ns))
Destination:     u_led_driver/u_counter/count_reg[25]/D
                  (rising edge-triggered cell FDCE clocked by sys_clk_pin  (rise@0.000ns fall@4.000ns period=8.000ns))

```

Le chemin le plus contraint relie la sortie de la bascule count_reg[6] à l'entrée de la bascule count_reg[25], toutes deux situées dans le compteur du counter_unit.

Ce chemin critique est la chaîne de retenue de l'incrémenteur 28 bits.

Question 15 :

(cf. Vidéo de démonstration)